



1. Heap Sort
2. Kth Largest Element
3. Kth Largest Element for all windows
4. Running Median

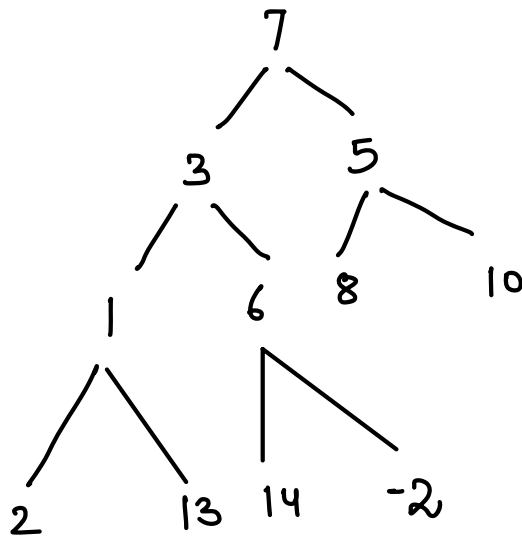


Q. Build heap in $O(N)$ time

Convert the given array $\longrightarrow$ into an array which follows heap properties $\longrightarrow$

1. CBT
2. HOP

[7, 3, 5, 1, 6, 8, 10, 2, 13, 14, -2]



1) Sort $\longrightarrow$ TC: $O(N \log N)$

2) [7, 3, 5, 1, 6, 8, 10, 2, 13, 14, -2]

$N \times$ list $[] = \{$

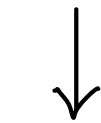
$\log N \leftarrow$ insert $\longrightarrow$ Heap Order Property, CBT

3) $\rightarrow$ Array $\dots \rightarrow$ Heap array $\dots \rightarrow O(N)$

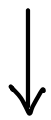
A = [-2 11 0 13 27 31 5 19]
 $\uparrow$
 N-1

HOP

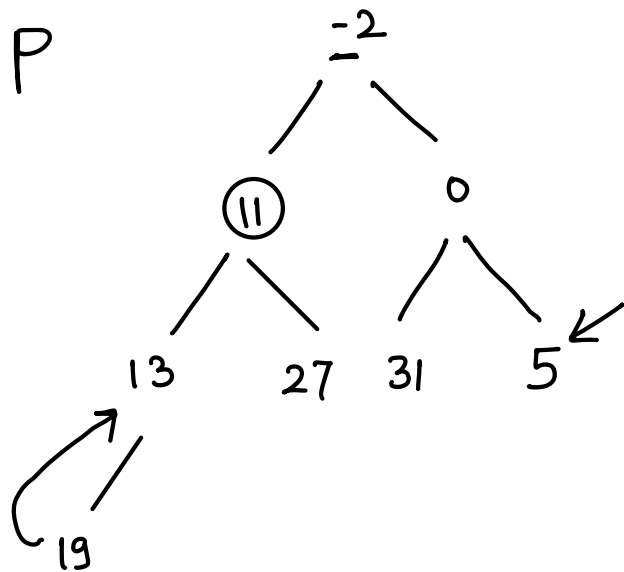
min
Heap



min
ele



Parent

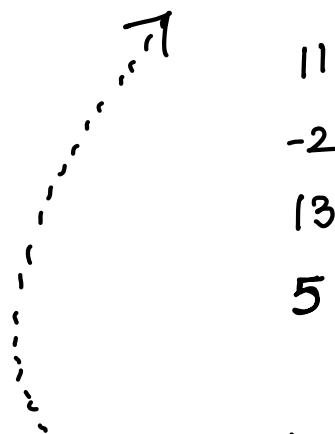


Upheapify

$\times$ root $\dots 1$

$\{ \text{Remaining} \} \times \log N$
 R_1

downheapify



downheapify

$\times$ leaf nodes $\dots$ so many

$\{ \text{Remaining} \} \times \log N$
 R_2

Down heapify process

$\rightarrow$ from first
 NON
 Leaf node

First NON Leaf node = Parent of last leaf node

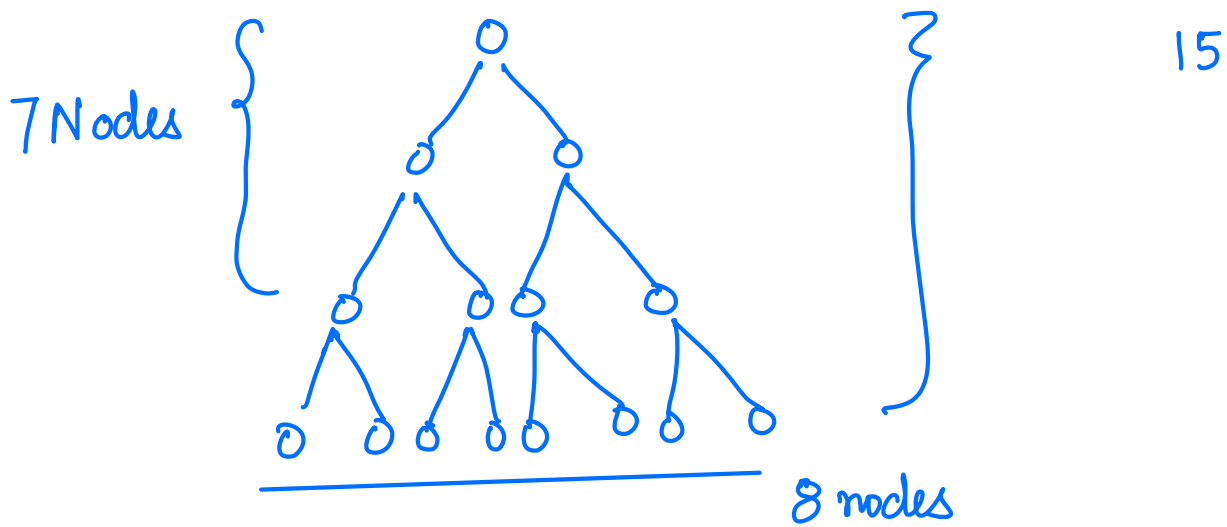
$$\begin{array}{ccc} \underline{i} & \longrightarrow & \frac{i-1}{2} \\ \text{Child} & & \text{Parent} \end{array}$$

$$\begin{aligned} N-1 & \longrightarrow \frac{(N-1)-1}{2} \\ & = \frac{N-2}{2} \quad \checkmark \end{aligned}$$

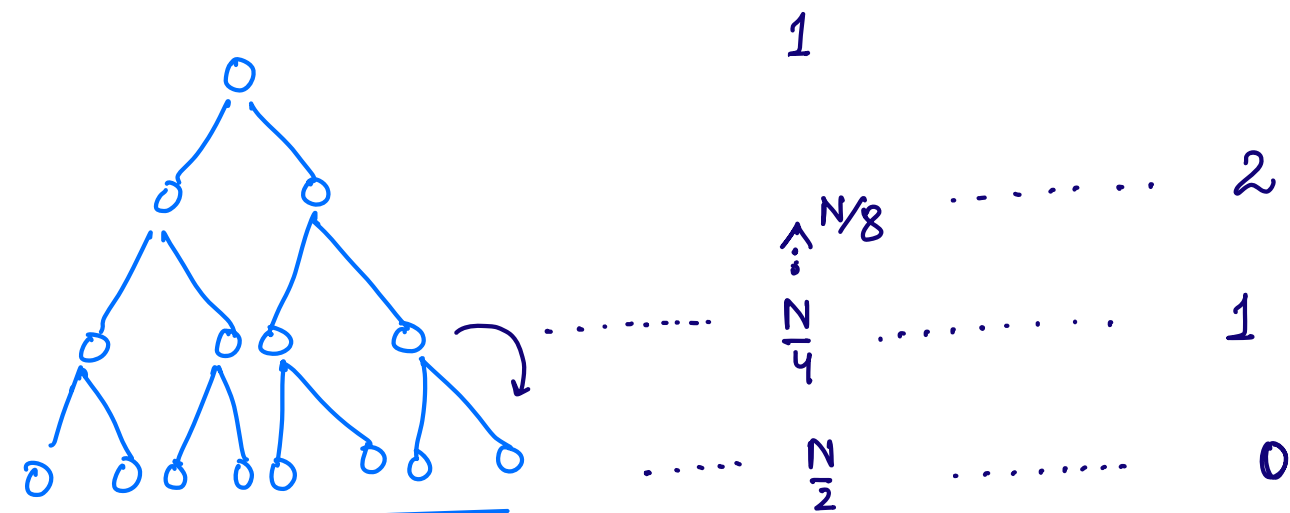
TC: $O(N)$

SC: $O(1)$

```
void convertToHeap ( int[] A ) {  
    N = A.length  
    for ( i = (N-2)/2 ; i >= 0 ; i-- ) {  
        downHeapify ( A, i );  
    }  
}
```



N nodes $\dots \rightarrow$ last level $\Rightarrow \frac{N}{2}$ nodes
effort



$$TC = \frac{N}{2} \times 0 + \frac{N}{4} \times 1 + \frac{N}{8} \times 2 + \frac{N}{16} \times 3 + \dots$$

$$= \frac{N}{4} + 2 \frac{N}{8} + \frac{3N}{16} + \frac{4N}{32} + \frac{5N}{64} + \dots$$

$$= \frac{N}{2} \left[\frac{1}{2} + \frac{2}{4} + \frac{3}{8} + \frac{4}{16} + \frac{5}{32} + \dots \right]$$

AGP

$$\downarrow$$

$$S = 2$$

$$\rightsquigarrow TC = \frac{N}{2} \times 2 = \underline{O(N)}$$

$$S = \frac{1}{2} + \frac{2}{4} + \frac{3}{8} + \frac{4}{16} + \frac{5}{32} \dots \dots \dots$$

$$- \frac{S}{2} = \frac{1}{4} + \frac{2}{8} + \frac{3}{16} + \frac{4}{32} + \dots \dots \dots$$

$$\frac{S}{2} = \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{16} + \frac{1}{32} \dots \dots \dots$$

↓
Pure GP

$$a = \frac{1}{2}$$

$$r = \frac{1}{2}$$

$$N = \infty \dots \dots \rightsquigarrow = \frac{a}{1-r} \quad (r < 1)$$

$$\frac{S}{2} = \frac{1/2}{1 - 1/2} = 1 \quad \left| \begin{array}{l} \Rightarrow \frac{S}{2} = 1 \\ \Rightarrow \boxed{S = 2} \end{array} \right.$$



Sort the array

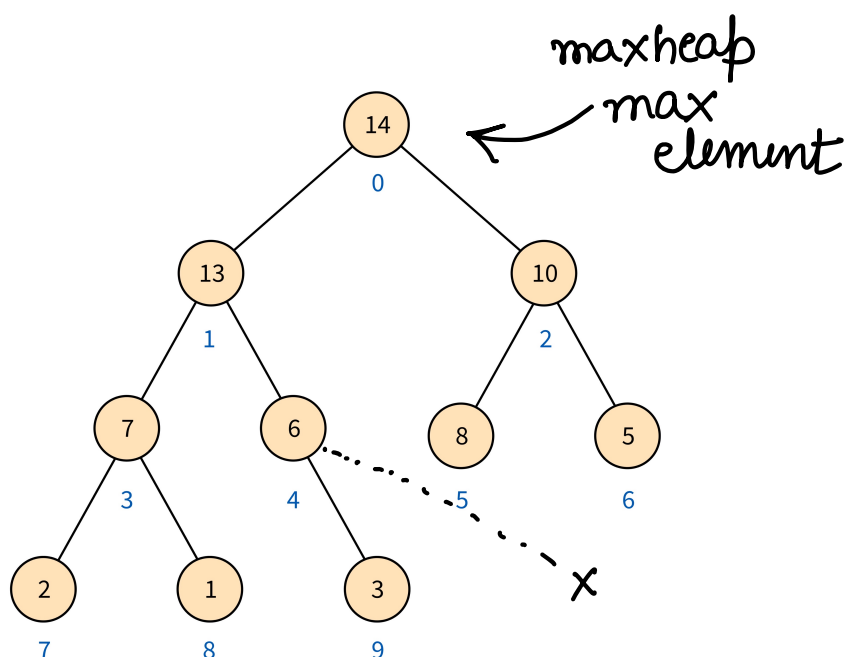
$$A = [13 \quad 11 \quad 0 \quad -2 \quad 27 \quad 31 \quad 5 \quad 19]$$



$$TC: O(N)$$

Array that follows HOP = $[-2 \quad 11 \quad 0 \quad 13 \quad 27 \quad 31 \quad 5 \quad 19]$

Can you sort the array?





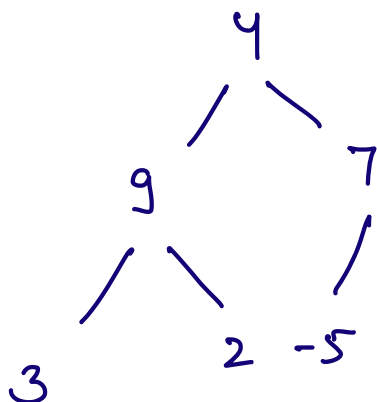
Heap - Sort

- 1) Convert array $\dots \rightarrow$ heap $\dots \rightarrow O(N)$
- 2) extract-min() $\dots \rightarrow$ Take 0th element
It is the minimum element

3) swap it with last element

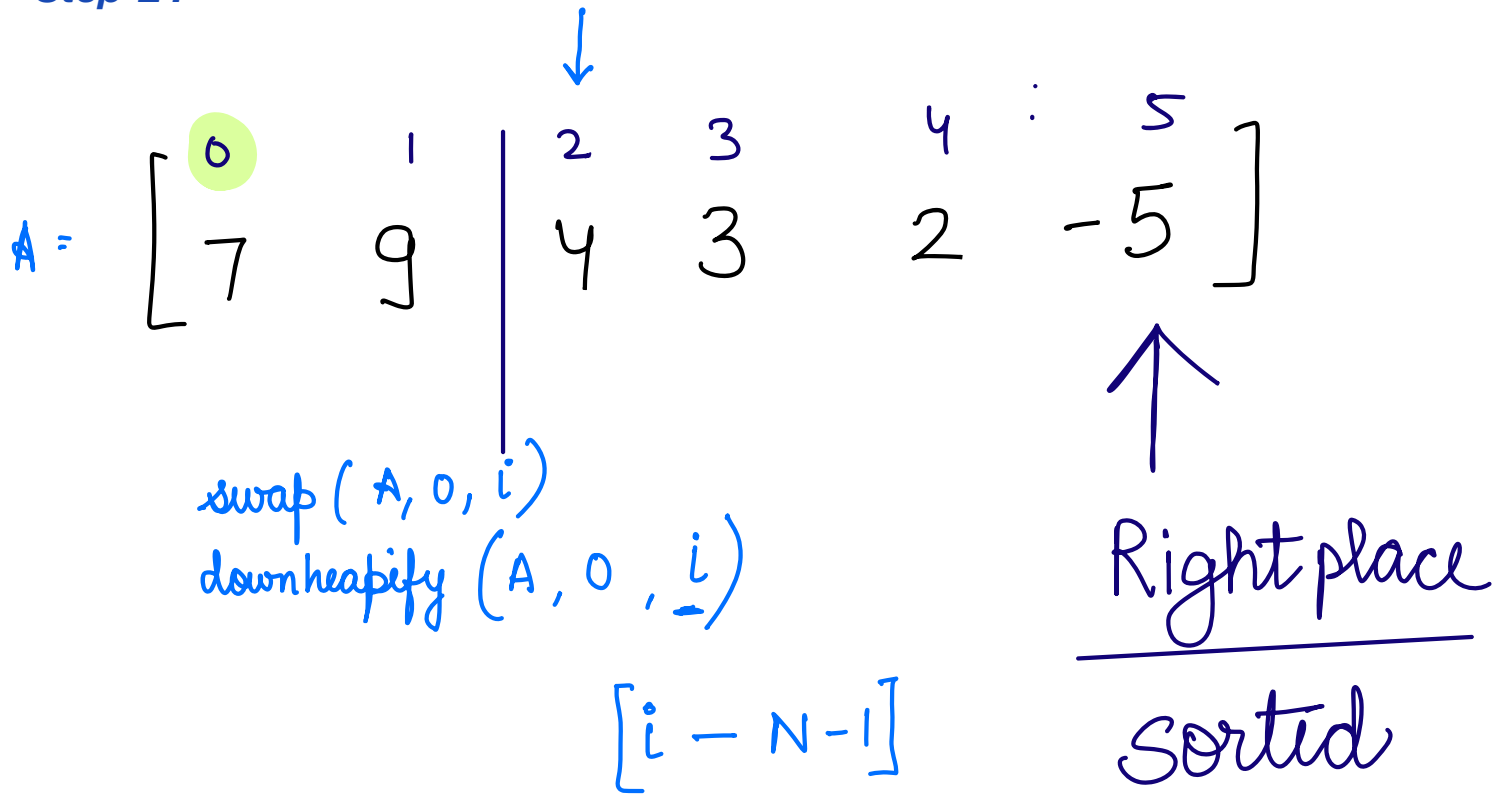
4) First/0th index element $\rightarrow$ downheapify

$\left[\begin{matrix} 0 & 1 & 2 & 3 & 4 & 5 \\ -5 & 2 & 3 & 9 & 7 & 4 \end{matrix} \right]$





Step-2 :



descending order

ascending order ... $\rightarrow$ Reverse

```
void heapsort ( int[] A ) {
    N = A.length
```

```
    for ( i = (N-2)/2; i >= 0; i-- ) { } N
        downheapify ( A, i, N )
    }
```

$N \log N$ {

```
    for ( int i = N-1; i >= 0; i-- ) {
        swap ( A, 0, i )
        downheapify ( A,  $\uparrow$ , i )
    }
```



N { descending order
 ↙ ↘
 reverse

TC: $O(N \log N)$
 SC: $O(1)$

}

```
void downheapify ( A[], i, idx ) {
```

```
    N = A.length
```

```
    while ( 2*i + 1 < idx ) {  
        x = min(heap[i], heap[2i+1],  
                heap[2i+2])
```

```
        if (x == heap[i]) {  
            break;
```

```
        } else if ( x == heap[2i+1] ) {  
            swap ( heap, i, 2i+1 )  
            i = 2i+1  
        }
```

```
        else {  
            swap ( heap, i, 2i+2 )  
            i = 2i+2
```

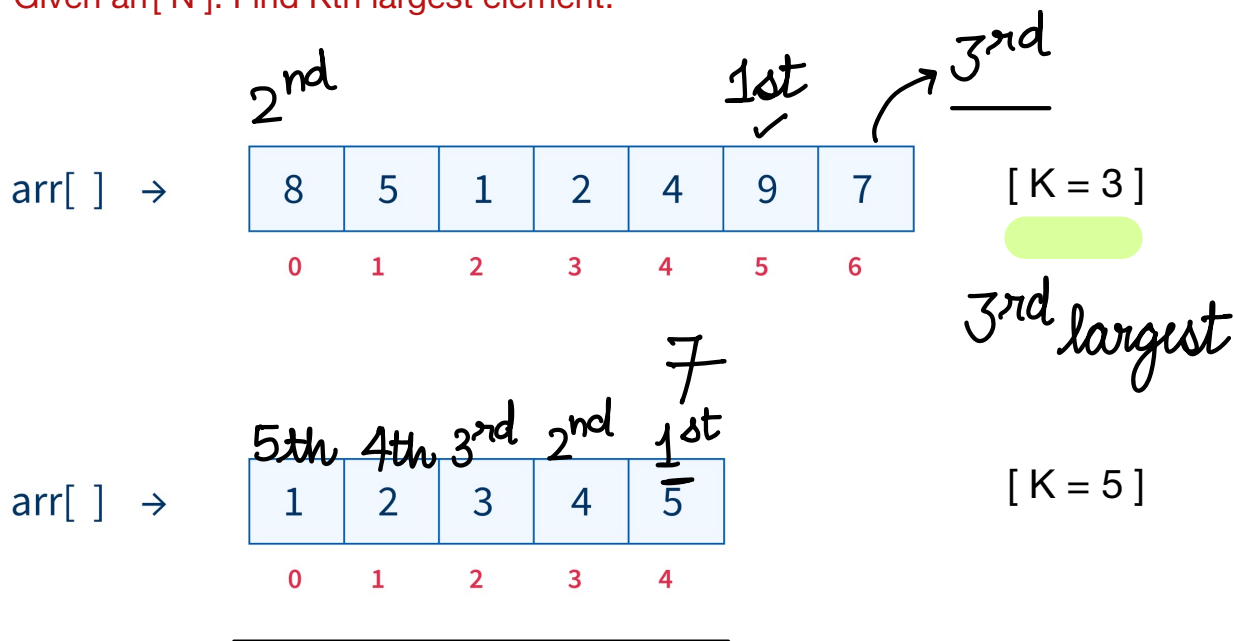
```
        }
```

```
    }
```

```
}
```



< Question > : Given arr[N]. Find Kth largest element.



💡 Idea -1

Sorting $\dots \rightarrow O(N \log N)$
 Sorted array $\dots \rightarrow \underline{\text{arr}[N-K]}$

💡 Idea -2

Heap sort

$\dots \rightarrow O(N + K \log N)$

$O(N)$. Build max heap and insert all the
 element max-heap

$(K \log N)$. extract-max() $\dots \dots \dots$ (K-1) times
 Kth largest $\dots \rightarrow$ K-1 extract max
 root

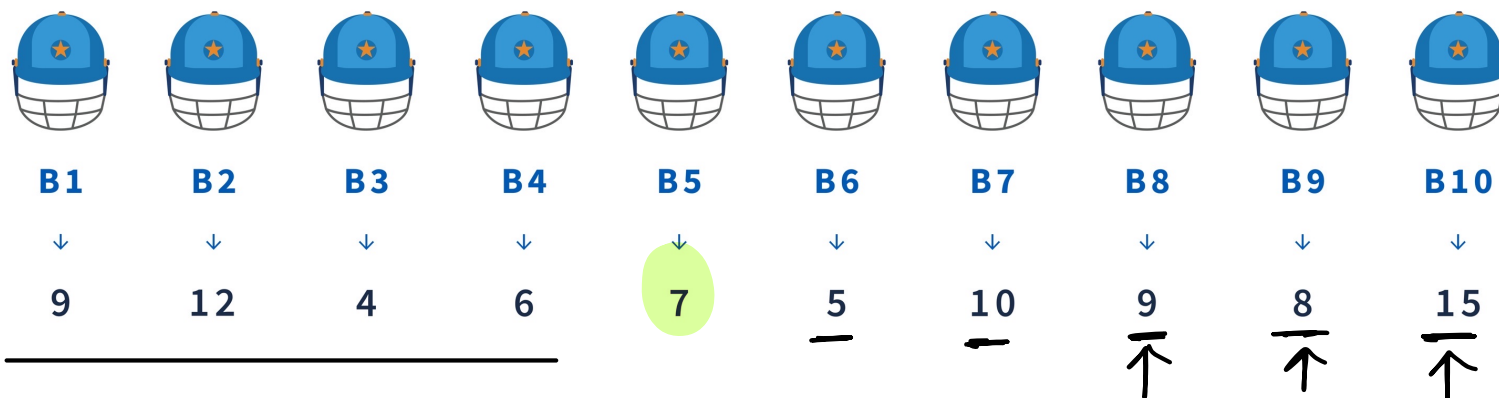
0
 1st — root
 2nd — 1 root
 3rd — 2 root



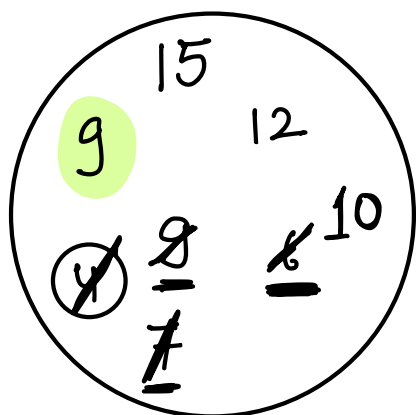
Idea -3

Team Selection

- Select 4 best batsman



best
4
bats
- man



min heap = root

i = K

4 size

min of these 4 people

to

i = N-1

</> Code

// Build min heap with first K elements [0 to K-1]

Iterate $i = K$ to $N-1 \dots \rightarrow$

```
[ if (arr[i] > min_element in heap) {  
    .extract_min()  
    .insert (arr[i])  
}
```

ans = extract_min()

return ans

TC: $O(K + (N-K) \log K)$

SC: $O(K)$